Bluetooth Packet Sniffing and Analysis Report

Student: Craig Ssemakula
Date: February 12, 2026
Tools Used: Wireshark with Bluetooth Virtual Sniffer (BTP)
Device Tested: JBL TUNE660NC Wireless Headphones

Introduction

This assignment involved installing the Bluetooth Test Platform (BTP) extension for Windows and using Wireshark to capture and analyze Bluetooth packets. The objective was to understand how Bluetooth devices communicate by observing packet-level traffic during real-world usage scenarios, specifically while streaming audio to wireless headphones.

Installation and Setup

The Bluetooth Test Platform Pack (version 1.14.0) was downloaded from Microsoft's official servers and installed on a Windows machine. The installation package includes the btvs.exe (Bluetooth Virtual Sniffer) utility located in C:\BTP\v1.14.0\x86\. Additionally, Wireshark was configured to work with the BTP extension to enable real-time Bluetooth packet capture.

Fig 1: Downloaded BTP installation package (BluetoothTestPlatformPack-1.14.0.msi)

Fig 2: Connected JBL TUNE660NC Bluetooth headphones in Windows

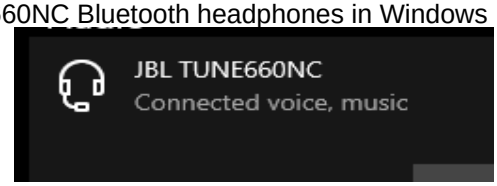
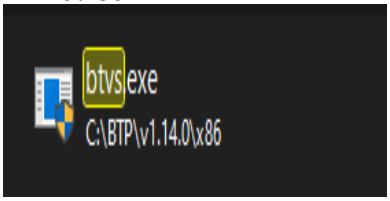


Fig 3: btvs.exe located in C:\BTP\v1.14.0\x86



Packet Capture Process

The Bluetooth Virtual Sniffer was launched and configured to interface with Wireshark. When the JBL TUNE660NC headphones were connected and actively streaming audio (music playback), Wireshark began capturing Bluetooth HCI (Host Controller Interface) packets in real-time. The capture included both control packets and data packets used for audio streaming.

Fig 4: Windows Bluetooth Virtual Sniffer status window showing active connection to Wireshark

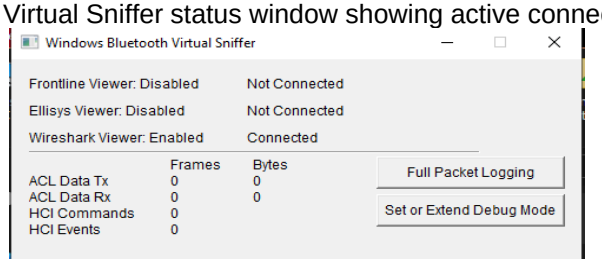
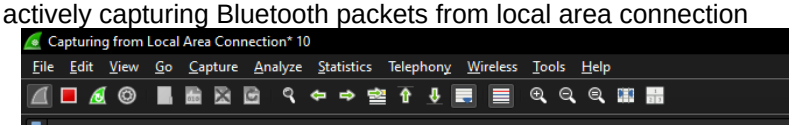


Fig 5: Wireshark actively capturing Bluetooth packets from local area connection



Packet Analysis and Observations

During the audio streaming session, Wireshark captured a substantial volume of Bluetooth traffic. The capture revealed several key packet types and communication patterns:

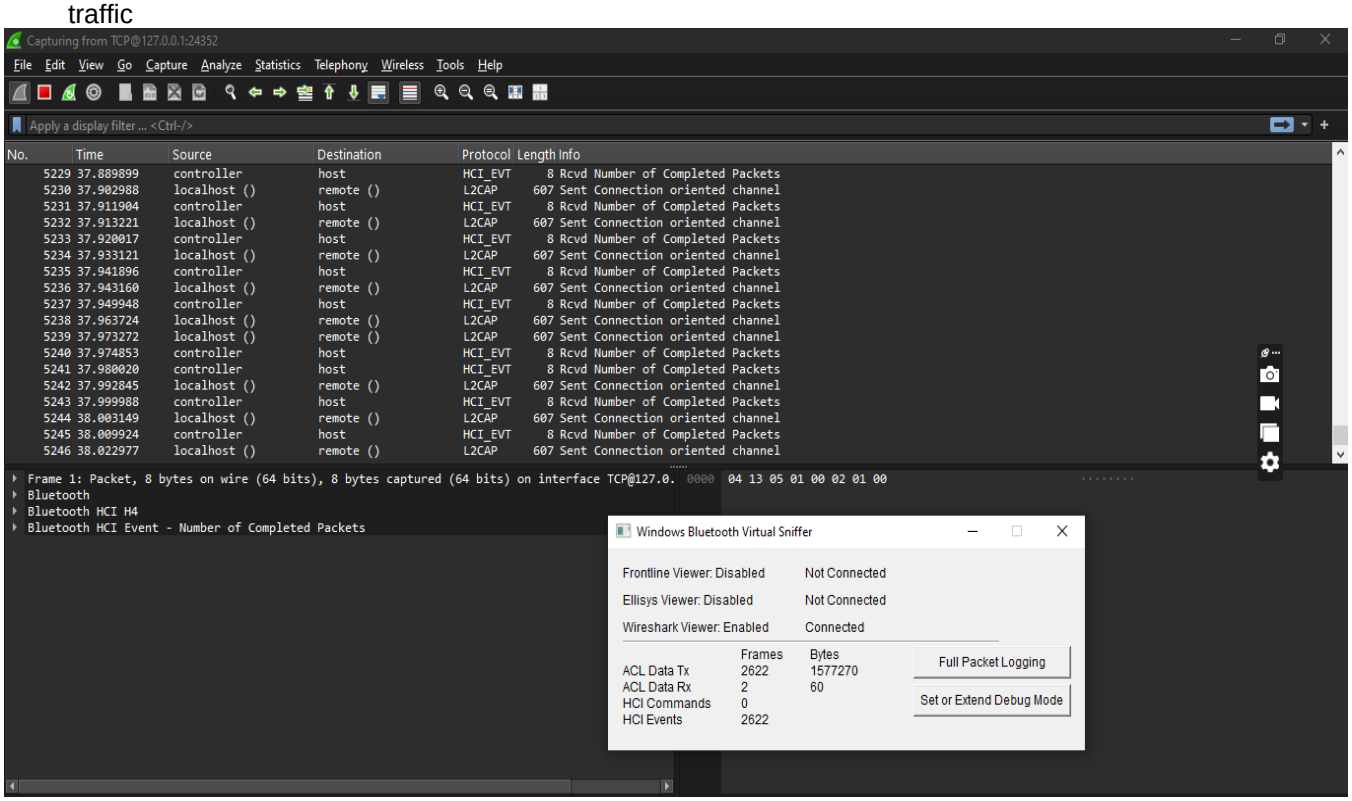
1. HCI Event Packets: The majority of captured packets were HCI_EVT (HCI Event) type packets, specifically "Number of Completed Packets" events. These events are sent by the Bluetooth controller to inform the host that data packets have been successfully transmitted. This is crucial for flow control in Bluetooth communication.

2. L2CAP Protocol: Connection-oriented channels were established using L2CAP (Logical Link Control and Adaptation Protocol), which provides higher-level data services. Multiple L2CAP packets with protocol 607 were observed, indicating channel setup and data transfer.

3. Connection Handle: The packets showed a connection handle of 0x0200, which uniquely identifies the Bluetooth connection between the computer and the JBL headphones. All packets related to this audio streaming session used this same handle.

4. High Packet Frequency: The capture statistics showed 2,622 frames transmitted (ACL Data Tx) and only 2 received (ACL Data Rx), totaling approximately 1.5 MB of data. This asymmetric traffic pattern is typical for audio streaming, where data flows predominantly from the computer to the headphones.

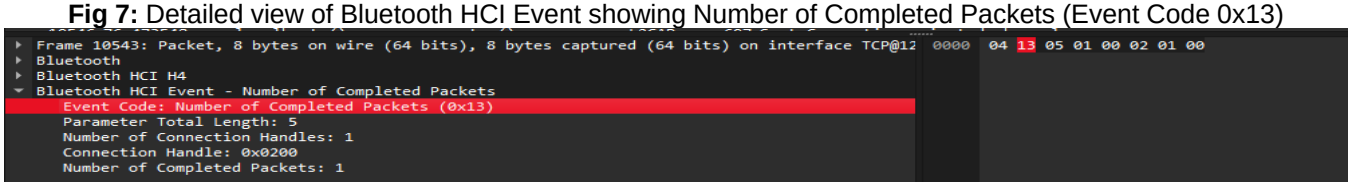
Fig 6: Wireshark packet list showing Bluetooth HCI events, L2CAP packets, and connection-oriented traffic



Detailed Packet Examination

By examining individual packets in depth, specific Bluetooth protocol mechanics became visible. The "Number of Completed Packets" event (Event Code 0x13) provides critical information about the Bluetooth transmission pipeline. Each event contains the connection handle, the total parameter length, and the count of successfully completed packets.

Fig 7: Detailed view of Bluetooth HCI Event showing Number of Completed Packets (Event Code 0x13)



In the detailed packet view shown above, Frame 10543 demonstrates a typical HCI Event packet structure: - **Event Code:** 0x13 (Number of Completed Packets) - **Parameter Length:** 5 bytes - **Connection Handle:** 0x0200 - **Completed Packets:** 1 packet successfully transmitted This event confirms that the Bluetooth controller successfully sent audio data to the headphones and is ready to accept more packets from the host, ensuring smooth audio playback without buffer overruns.

Key Learnings

This assignment provided valuable insights into Bluetooth communication at the packet level. Key takeaways include:

Protocol Layering: Bluetooth uses a layered architecture with HCI for hardware control and L2CAP for logical data channels.

Flow Control Mechanism: The 'Number of Completed Packets' event is essential for managing data flow and preventing buffer overflow.

Audio Streaming Characteristics: Bluetooth audio streaming generates high-frequency, asymmetric traffic patterns with continuous data transmission from host to device.

Packet Structure: Each Bluetooth packet contains specific fields including connection handles, event codes, and protocol-specific parameters that enable reliable communication.

Analysis Tools: Wireshark combined with BTP provides powerful capabilities for deep Bluetooth protocol analysis and troubleshooting.

Conclusion

Successfully capturing and analyzing Bluetooth packets revealed the underlying mechanics of wireless audio transmission. The exercise demonstrated how Bluetooth maintains reliable connections through carefully managed packet flow and event-driven communication. Understanding these low-level protocols is crucial for debugging wireless connectivity issues, optimizing Bluetooth applications, and developing new Bluetooth-enabled devices.